

สารบัญ.....	Error! Bookmark not defined.
ส่วนที่ 1: ข้อมูลรายวิชาและวัตถุประสงค์การเรียนรู้.....	3
1.1 วัตถุประสงค์การเรียนรู้ (Learning Objectives — ตาม Bloom’s Revised Taxonomy).....	3
ส่วนที่ 2: บทนำ — ทำไมต้องจำลองก่อนต่อวงจรจริง.....	4
ส่วนที่ 3: รู้จัก Wokwi Simulator.....	4
3.1 Wokwi คืออะไร.....	4
3.2 ขอบเขตการรองรับ.....	5
3.3 แผนการใช้งานและค่าบริการ.....	5
3.4 ข้อจำกัดที่ต้องเน้นย้ำกับผู้เรียน.....	6
ส่วนที่ 4: การติดตั้งและตั้งค่า Wokwi Extension บน VS Code.....	6
4.1 ขั้นตอนการติดตั้ง.....	6
4.2 สถาปัตยกรรมการทำงาน — สิ่งที่นักศึกษามักเข้าใจผิด.....	7
ส่วนที่ 5: โครงสร้างโปรเจกต์และไฟล์ตั้งค่าสำคัญ.....	7
5.1 ไฟล์ wokwi.toml — บอกว่าเฟิร์มแวร์อยู่ที่ไหน.....	7
5.2 ไฟล์ diagram.json — วงจรในรูปแบบข้อความ.....	8
ส่วนที่ 6: การติดตั้ง Toolchain สำหรับคอมไพล์.....	9
6.1 Arduino CLI — สำหรับโปรเจกต์ Arduino Sketch.....	9
6.2 PlatformIO Core — สำหรับงานหลายแพลตฟอร์ม.....	10
6.3 GCC-AVR Toolchain — สำหรับเขียนระดับรีจิสเตอร์.....	10
ส่วนที่ 7: ปฏิบัติการที่ 1 — Arduino Uno กับจอ LCD I2C.....	10
7.1 แบบฝึกในคาบ — เพิ่มการนับและส่งข้อความออก Serial.....	11
7.2 การตั้งค่า tasks.json เพื่อคอมไพล์ด้วยปุ่มลัด.....	12
ส่วนที่ 8: ปฏิบัติการที่ 2 — AVR LibC กับ ADC และ PWM.....	12
8.1 โครงสร้างโปรเจกต์และคำสั่งคอมไพล์.....	12
8.2 โค้ดตัวอย่าง.....	13
ส่วนที่ 9: ปฏิบัติการที่ 3 — ESP32 กับ FreeRTOS.....	14

9.1 การสร้างโปรเจกต์ด้วย PlatformIO	14
9.2 โค้ดตัวอย่างการสร้างทาสก์	14
ส่วนที่ 10: การบูรณาการ Generative AI ในงานพัฒนาระบบฝังตัว	15
10.1 หลักคิด: AI เป็นผู้ช่วยดีบัก ไม่ใช่ผู้เขียนแทน	15
10.2 Workflow การใช้ AI ในปฏิบัติการนี้ (5 ขั้นตอน)	16
10.3 ตัวอย่าง Prompt ที่ใช้ได้จริง	16
ส่วนที่ 11: จริยธรรมและความซื่อสัตย์ทางวิชาการ	17
ส่วนที่ 12: กิจกรรมในชั้นเรียน (In-class Activities)	17
ส่วนที่ 13: แบบฝึกหัด/คำถามท้ายบท (พร้อมแนวคำตอบ)	18
13.1 คำถามทบทวน	18
13.2 แบบฝึกหัดปฏิบัติ (ส่งเป็นชิ้นงาน)	19
ส่วนที่ 14: เกณฑ์การประเมิน (Assessment Rubric)	19
ส่วนที่ 15: เอกสารอ้างอิงและแหล่งค้นคว้าเพิ่มเติม	20

ส่วนที่ 1: ข้อมูลและวัตถุประสงค์การเรียนรู้

รายการ	รายละเอียด
หัวข้อ	Wokwi Simulator + VS Code IDE สำหรับการพัฒนาระบบสมองกลฝังตัวและ IoT
กลุ่มเป้าหมาย	นักศึกษาปริญญาตรี เทคโนโลยีสารสนเทศ / วิทยาการคอมพิวเตอร์ / IoT และระบบสมองกลฝังตัว
จำนวนคาบ	3-4 ชั่วโมง (บรรยาย 1 ชม. + ปฏิบัติการ 2-3 ชม.)
ความรู้พื้นฐาน	การเขียนโปรแกรมภาษา C/C++ เบื้องต้น, การใช้คำสั่งพื้นฐานบน Terminal, หลักการวงจรดิจิทัลและอิเล็กทรอนิกส์เบื้องต้น
อุปกรณ์ที่ต้องเตรียม	เครื่องคอมพิวเตอร์ (Windows + WSL2 Ubuntu / Ubuntu / macOS), VS Code, บัญชี Wokwi (สมัครฟรี), อินเทอร์เน็ต

1.1 วัตถุประสงค์การเรียนรู้ (Learning Objectives — ตาม Bloom’s Revised Taxonomy)

เมื่อเรียนจบบทเรียนนี้ นักศึกษาจะสามารถ:

- (Remember)** ระบุนามความสามารถและข้อจำกัดของ Wokwi Simulator รวมถึงหน้าที่ของไฟล์ `wokwi.toml` และ `diagram.json` ในโปรเจกต์ได้
- (Understand)**
อธิบายความแตกต่างระหว่างการจำลองผ่านเว็บเบราว์เซอร์กับการคอมไพล์บนเครื่องแล้วจำลองผ่าน VS Code รวมถึงบทบาทของ Arduino CLI และ PlatformIO Core ได้
- (Apply)** ติดตั้ง Wokwi Extension ขอ License Key และคอมไพล์โปรเจกต์ Arduino ด้วย `arduino-cli` หรือ `pio` จนจำลองการทำงานสำเร็จได้
- (Analyze)** วิเคราะห์ข้อความผิดพลาด (error/warning) จากคอมไพเลอร์และผลการจำลอง เพื่อระบุสาเหตุที่แท้จริงของปัญหาได้
- (Evaluate)** ประเมินความถูกต้องของโค้ดและวงจรที่ได้จาก Generative AI เทียบกับเอกสารข้อกำหนดของอุปกรณ์ (datasheet) และผลการจำลองจริงได้
- (Create)** ออกแบบและสร้างโปรเจกต์ต้นฉบับที่ประกอบด้วยไฟล์ `diagram.json`, `wokwi.toml`, ซอร์สโค้ด และไฟล์ตั้งค่าการคอมไพล์ พร้อมสาธิตการทำงานได้

ส่วนที่ 2: บทนำ — ทำไมต้องจำลองก่อนต่อวงจรจริง

การเรียนรู้ระบบสมองกลฝังตัว (Embedded Systems) และ IoT ตามปกติต้องอาศัยฮาร์ดแวร์จริง ซึ่งสร้างอุปสรรคหลายด้านต่อผู้เรียน ได้แก่ **ต้นทุนอุปกรณ์**, **ความเสี่ยงที่อุปกรณ์เสียหาย** จากการต่อวงจรผิด, และ **ข้อจำกัดด้านเวลาและสถานที่** เพราะต้องอยู่ในห้องปฏิบัติการเท่านั้น เครื่องมือจำลอง (Simulator) จึงเข้ามาเติมช่องว่างนี้ โดยให้ผู้เรียนทดลองตรรกะของโปรแกรมและการเชื่อมต่อวงจรได้ทุกที่ทุกเวลา ก่อนนำไปทดสอบกับอุปกรณ์จริง

บทเรียนนี้เน้นรูปแบบการทำงานที่ต่างจากการใช้ Wokwi ผ่านเว็บเบราว์เซอร์ทั่วไป นั่นคือการใช้ **VS Code IDE เป็นศูนย์กลางการพัฒนา** โดย **คอมไพล์โค้ดบนเครื่องของผู้เรียนเอง**

ด้วยเครื่องมือมาตรฐานอุตสาหกรรม (Arduino CLI, PlatformIO, GCC-AVR)

แล้วส่งเฉพาะไฟล์เฟิร์มแวร์ที่ได้ไปให้ Wokwi จำลองการทำงาน

วิธีนี้ให้ประโยชน์สามข้อที่สำคัญต่อการเรียนวิชาชีพ:

- **ความเร็ว** — ไม่ต้องรอคิวคอมไพล์บนเซิร์ฟเวอร์ การแก้ไขแล้วทดสอบซ้ำจึงทำได้เร็วกว่ามาก
- **เครื่องมือมืออาชีพ** — ผู้เรียนได้ฝึกใช้ Git, Terminal, ระบบจัดการไลบรารี และ toolchain ชุดเดียวกับที่ใช้ในงานจริง ไม่ใช่เฉพาะหน้าเว็บสำเร็จรูป
- **การควบคุมโปรเจกต์** — โครงสร้างไฟล์ทั้งหมดอยู่ในเครื่อง จึงบริหารเวอร์ชันด้วย Git และส่งงานเป็นรีโพสิทอรีได้

เอกสารนี้เรียบเรียงจาก: บทความ "แนะนำการใช้งาน VS Code IDE + Wokwi" จากเว็บไซต์ IoT Engineering Education (iot-kmutnb.github.io) ซึ่งเผยแพร่ภายใต้สัญญาอนุญาต Creative Commons Attribution-ShareAlike 4.0 International โดยผู้สอนได้ขยายเนื้อหา ปรับปรุงข้อมูลเวอร์ชันให้เป็นปัจจุบัน (สิงหาคม 2026) และเพิ่มองค์ประกอบการเรียนการสอน

ส่วนที่ 3: รู้จัก Wokwi Simulator

3.1 Wokwi คืออะไร

Wokwi เป็นซอฟต์แวร์ประเภท Web Application

สำหรับเขียนโค้ดและจำลองการทำงานของไมโครคอนโทรลเลอร์

ทำงานผ่านเว็บเบราว์เซอร์โดยตรงโดยผู้ใช้อาจไม่ต้องติดตั้งโปรแกรมลงเครื่อง เริ่มพัฒนาในปี ค.ศ. 2019 โดย **Uri Shaked** นักพัฒนาซอฟต์แวร์ชาวอิสราเอล

จุดประสงค์หลักคือให้ผู้ใช้ทดสอบและพัฒนาโค้ดได้โดยไม่ต้องมีฮาร์ดแวร์จริง
จึงเหมาะกับการเรียนรู้และฝึกเขียนโปรแกรมไมโครคอนโทรลเลอร์ในเบื้องต้น

3.2 ขอบเขตการรองรับ

ด้าน	สิ่งที่รองรับ
ภาษา/แพลตฟอร์ม	Arduino (C/C++), MicroPython, Rust, C/C++, Verilog (สำหรับงานออกแบบวงจรระดับดิจิทัล)
บอร์ดตระกูล Arduino	Arduino Uno, Arduino Nano, Arduino Mega 2560
บอร์ดอื่น ๆ	ESP32 / ESP32-S3 / ESP32-C3, Raspberry Pi Pico, STM32 Nucleo C031, STM32 Nucleo32 L031K6
อุปกรณ์รอบข้าง	LED, ปุ่มกด, จอ LCD 16x2 แบบ I2C, จอ OLED I2C, โฟตอนซีอิมิเตอร์, Logic Analyzer และอื่น ๆ (ดูรายการเต็มที่ elements.wokwi.com)
เครื่องมือประกอบ	Serial Monitor สำหรับตรวจสอบผลลัพธ์, `wokwi-cli` สำหรับสั่งงานแบบ Command Line, Wokwi for VS Code สำหรับพัฒนาบนเครื่องตนเอง

รายการฮาร์ดแวร์ที่รองรับทั้งหมดตรวจสอบได้จาก docs.wokwi.com/getting-started/supported-hardware ซึ่งมีการเพิ่มอุปกรณ์ใหม่อยู่เสมอ
ผู้สอนควรให้นักศึกษาตรวจหน้านี้ก่อนออกแบบโปรเจกต์ทุกครั้ง

3.3 แผนการใช้งานและค่าบริการ

Wokwi เปิดให้ใช้งานทั้งแบบฟรีและแบบมีค่าใช้จ่ายรายเดือน จำแนกได้ดังนี้ (อ้างอิงหน้า
wokwi.com/pricing):

- **Community plan (ฟรี)** — สำหรับงานส่วนตัวและโครงการเปิดเผยซอร์ส
สร้างโปรเจกต์สาธารณะได้ไม่จำกัด เหมาะกับงานในรายวิชานี้
- **Hobby และ Hobby+** — สำหรับผู้ใช้งานส่วนตัวขั้นสูงขึ้นไป มีค่าใช้จ่ายรายเดือน
- **Pro** — สำหรับโปรเจกต์เชิงพาณิชย์หรือผู้ที่ต้องการสมรรถนะสูงขึ้น
- **University Classroom** — แผนสำหรับสถาบันการศึกษา
ซึ่งอาจารย์ผู้สอนสามารถติดต่อขอใช้สำหรับการจัดการเรียนการสอนทั้งชั้นเรียนได้

3.4 ข้อจำกัดที่ต้องเน้นย้ำกับผู้เรียน

คำเตือนสำคัญ: Wokwi ใช้ทดสอบโค้ดด้วยการจำลองการทำงานของไมโครคอนโทรลเลอร์ได้ดี แต่ฟังก์ชันการทำงานของฮาร์ดแวร์บางส่วนอาจยังไม่รองรับหรือไม่ครอบคลุมทั้งหมด ดังนั้นผลการจำลองที่ "ผ่าน" ไม่ได้รับประกันว่าวงจรจริงจะทำงานได้ ประเด็นที่ตัวจำลองมักไม่สะท้อน ได้แก่ ความคลาดเคลื่อนของสัญญาณนาฬิกา สัญญาณรบกวน (noise) กระแสไฟฟ้าที่อุปกรณ์ดึงจริง ปัญหาการต่อกราวด์ และการเสื่อมสภาพของอุปกรณ์

ส่วนที่ 4: การติดตั้งและตั้งค่า Wokwi Extension บน VS Code

4.1 ขั้นตอนการติดตั้ง

1. **ติดตั้ง VS Code** — ดาวน์โหลด Visual Studio Code จากเว็บไซต์ทางการของ Microsoft และติดตั้งให้เรียบร้อยก่อน
2. **เปิดใช้งาน VS Code** — กรณีใช้ Ubuntu โดยตรง เปิดจากเมนูแอปพลิเคชัน หรือพิมพ์คำสั่ง `code` ใน Terminal
3. **กรณีใช้ WSL2 Ubuntu บน Windows** — เปิด VS Code ในโหมด Remote - WSL เพื่อให้เชื่อมต่อกับสภาพแวดล้อม Ubuntu โดยตรง ส่วนขยายและ toolchain ทั้งหมดจะถูกติดตั้งฝั่ง Ubuntu
4. **ค้นหาและติดตั้ง Wokwi Extension** — ในแถบ Extensions ค้นหาคำว่า "Wokwi Simulator" แล้วกดติดตั้ง
5. **ขอ License Key** — กด F1 (หรือ Ctrl+Shift+P) แล้วเลือกคำสั่ง "Wokwi: Request a new License" ระบบจะเปิดเว็บเบราว์เซอร์ให้ยืนยัน จากนั้นกดปุ่มขอ License และอนุญาตให้ส่งกลับเข้า VS Code เมื่อสำเร็จจะขึ้นข้อความยืนยันการเปิดใช้งาน

หากเบราว์เซอร์ไม่ส่ง License กลับให้อัตโนมัติ ให้คัดลอกคีย์ที่แสดงบนหน้าเว็บ แล้วใช้คำสั่ง `Wokwi: Manually Enter License Key` ใน Command Palette เพื่อวางคีย์เอง ทำเพียงครั้งเดียวต่อเครื่อง

ข้อควรทราบเรื่องอายุ License: แหล่งข้อมูลต่าง ๆ ระบุอายุของ License ฟรีไม่ตรงกัน (บางแหล่งระบุ 30 วัน บางแหล่งระบุ 1 ปี) และเงื่อนไขอาจเปลี่ยนแปลงได้ ให้ตรวจสอบวันหมดอายุจริงจากหน้า wokwi.com/license และแนะนำนักศึกษาว่าเมื่อหมดอายุสามารถขอใหม่ได้ฟรีด้วยขั้นตอนเดิม

ข้อสังเกต: การใช้งาน Wokwi Simulator ร่วมกับ VS Code ใช้ได้กับระบบปฏิบัติการอื่นเช่นกัน ทั้ง Windows และ macOS ไม่จำกัดเฉพาะ Linux นอกจากนี้ Extension ยังใช้ร่วมกับเฟรมเวิร์กอื่นได้หลากหลาย เช่น ESP-IDF, Zephyr Project, Pi Pico SDK, Rust และ MicroPython

4.2 สถาปัตยกรรมการทำงาน — สิ่งที่น่าสนใจมักเข้าใจผิด

การจำลองยังทำงานบนเซิร์ฟเวอร์ของ Wokwi สิ่งที่ย้ายมาอยู่บนเครื่องผู้เรียนคือ "ขั้นตอนการคอมไพล์" เท่านั้น ดังนั้นจึงยังต้องเชื่อมต่ออินเทอร์เน็ตขณะจำลอง ลำดับการทำงานคือ:

```
ซอร์สโค้ด (.ino / .c / .cpp)
|
| คอมไพล์บนเครื่องผู้เรียน (arduino-cli / pio / avr-gcc)
v
ไฟล์เฟิร์มแวร์ (.hex / .bin) + ไฟล์สัญลักษณ์ (.elf)
|
| wokwi.toml ชี้ตำแหน่งไฟล์ให้ Extension
v
Wokwi Extension ---> จำลองการทำงานตามวงจรใน diagram.json
```

ส่วนที่ 5: โครงสร้างโปรเจกต์และไฟล์ตั้งค่าสำคัญ

โปรเจกต์ Wokwi ที่ทำงานบน VS Code ต้องมีไฟล์ตั้งค่าสองไฟล์เป็นอย่างน้อย ซึ่งเป็นหัวใจของบทเรียนนี้

5.1 ไฟล์ wokwi.toml — บอกว่าเฟิร์มแวร์อยู่ที่ไหน

ไฟล์นี้ใช้ตั้งค่าการทำงานร่วมกับ Wokwi Simulator โดยระบุตำแหน่งไฟล์ .hex หรือ .bin และไฟล์ .elf ที่ได้จากการคอมไพล์ ตำแหน่งไฟล์จะต่างกันตามเครื่องมือที่ใช้ ดังตาราง

เครื่องมือคอมไพล์	ค่าที่ต้องระบุใน wokwi.toml
arduino-cli (Uno)	firmware = 'build/arduino.avr.uno/<ชื่อโปรเจกต์>.ino.hex' · elf = 'build/arduino.avr.uno/<ชื่อโปรเจกต์>.ino.elf'
PlatformIO (Uno)	firmware = '.pio/build/uno/firmware.hex' · elf = '.pio/build/uno/firmware.elf'
PlatformIO (ESP32)	firmware = '.pio/build/esp32dev/firmware.bin' · elf = '.pio/build/esp32dev/firmware.elf'
avr-gcc (AVR LibC)	firmware = 'build/main.hex' · elf = 'build/main.elf'

สังเกตความแตกต่าง: บอร์ดตระกูล AVR ใช้ไฟล์ .hex ส่วน ESP32 ใช้ .bin และไฟล์ .elf

จำเป็นสำหรับการดีบั๊ก (แสดงชื่อฟังก์ชันและตัวแปร) —

ปัญหาที่พบบ่อยที่สุดของนักศึกษาคือระบุพารามิเตอร์หรือลิมิตคอมไพล์ก่อนกดจำลอง

5.2 ไฟล์ diagram.json — วงจรในรูปแบบข้อความ

ไฟล์นี้อธิบายวงจรทั้งหมดเป็นโครงสร้าง JSON ประกอบด้วยสองส่วนหลัก คือ รายการอุปกรณ์ (parts)

และรายการการเชื่อมต่อ (connections) ตัวอย่างวงจร LED ต่อกับขา D8 ของ Arduino Uno

ผ่านตัวต้านทานจำกัดกระแส:

```
{
  "version": 1,
  "author": "student",
  "editor": "wokwi",
  "parts": [
    { "type": "wokwi-arduino-uno", "id": "uno", "top": 0, "left": 0 },
    { "type": "wokwi-led", "id": "led1", "top": -80, "left": 120,
      "attrs": { "color": "green" } },
    { "type": "wokwi-resistor", "id": "r1", "top": -20, "left": 90,
      "attrs": { "value": "220" } }
  ],
  "connections": [
    [ "uno:8", "led1:A", "green", [ "v-20", "h-20" ] ],
    [ "led1:C", "r1:2", "green", [ "v0" ] ],
    [ "uno:GND.1", "r1:1", "black", [ "v-20", "h-20" ] ]
  ],
  "dependencies": {}
}
```

อ่านโครงสร้างนี้ให้ออกคือทักษะสำคัญ องค์กรประกอบที่ต้องเข้าใจมีดังนี้:

- **type** — ชนิดอุปกรณ์ตามรายการใน elements.wokwi.com เช่น `wokwi-led`, `wokwi-lcd1602`, `wokwi-potentiometer`
- **id** — ชื่อเรียกอุปกรณ์ในไฟล์นี้ ใช้อ้างอิงตอนเชื่อมต่อสาย ต้องไม่ซ้ำกัน
- **top / left** — ตำแหน่งวางบนผังวงจร (หน่วยพิกเซล) ไม่มีผลต่อการทำงานทางไฟฟ้า
- **attrs** — คุณสมบัติเฉพาะอุปกรณ์ เช่น สีของ LED ค่าความต้านทาน หรือโหมดการเชื่อมต่อของจอ LCD (`pins`: "i2c")
- **connections** — แต่ละรายการมีรูปแบบ [ขาด้านทาง, ขาปลายทาง, สีสาย, เส้นทางเดินสาย] โดยเส้นทางเป็นคำสั่งย่อ เช่น `v-20` = เดินขึ้น 20 หน่วย, `h30` = เดินขวา 30 หน่วย

เคล็ดลับ: การเชื่อมต่อกับ Serial Monitor ทำได้โดยเชื่อมต่อ TX/RX เข้ากับอุปกรณ์พิเศษชื่อ

\$serialMonitor เช่น ["esp:TX0", "\$serialMonitor:RX", "", []]

ส่วนที่ 6: การติดตั้ง Toolchain สำหรับคอมไพล์

การใช้ Wokwi ร่วมกับ VS Code จำเป็นต้องมีเครื่องมือคอมไพล์บนเครื่อง ในบทเรียนนี้จะใช้ 3 ทางเลือกตามระดับความลึกของงาน

6.1 Arduino CLI — สำหรับโปรเจกต์ Arduino Sketch

Arduino CLI เป็นเครื่องมือแบบ Command Line ที่ใช้คอมไพล์โค้ด Arduino Sketch ใช้ได้กับหลายระบบปฏิบัติการ ขั้นตอนติดตั้งบน Ubuntu / WSL2 Ubuntu มีดังนี้

```
# ดาวน์โหลดและติดตั้งด้วยสคริปต์ทางการ (ไฟล์จะถูกวางไว้ที่ ./bin)
curl -fsSL \
  https://raw.githubusercontent.com/arduino/arduino-cli/master/install.sh |
sh

mkdir -p $HOME/bin/
mv bin/arduino-cli $HOME/bin/

# เพิ่ม $HOME/bin เข้า PATH เพื่อให้เรียกคำสั่งได้จากทุกไคลเอนต์เทอร์มินัล
echo 'export PATH="$PATH:$HOME/bin"' >> $HOME/.bashrc
source $HOME/.bashrc

# ตรวจสอบเวอร์ชันที่ติดตั้ง
arduino-cli version

# สร้างไฟล์ตั้งค่า แล้วอัปเดตรายการบอร์ด
arduino-cli config init
arduino-cli core update-index

# ติดตั้งแกนบอร์ดตระกูล AVR (Uno, Nano, Mega)
arduino-cli core install arduino:avr
arduino-cli core list
```

การติดตั้งไลบรารีที่ต้องใช้ในบทเรียนนี้:

```
arduino-cli lib update-index
arduino-cli lib install "LiquidCrystal I2C"
arduino-cli lib install "Adafruit SSD1306"
arduino-cli lib install "Adafruit GFX Library"
```

ข้อมูลอัปเดต (สิงหาคม 2026): เวอร์ชันเสถียรล่าสุดของ Arduino CLI คือ 1.5.1 (เผยแพร่ 5 มิถุนายน 2026) ซึ่งใหม่กว่าเวอร์ชัน 1.4.0 ที่ปรากฏในเอกสารอ้างอิงต้นทาง

เนื่องจากสคริปต์ติดตั้งดึงเวอร์ชันล่าสุดเสมอ ผลลัพธ์ของคำสั่ง `arduino-cli version` ที่นักศึกษาเห็นจึงอาจไม่ตรงกับตัวอย่างในเอกสาร — ให้ถือเป็นเรื่องปกติ ไม่ใช่ข้อผิดพลาด

6.2 PlatformIO Core — สำหรับงานหลายแพลตฟอร์ม

PlatformIO เป็นเครื่องมือพัฒนาซอฟต์แวร์ฝังตัวที่รองรับบอร์ดหลายรุ่นและหลายเฟรมเวิร์ก เช่น Arduino, ESP-IDF และ STM32Cube ส่วน PlatformIO Core คือส่วนประกอบหลักที่ทำงานแบบ Command Line

```
sudo apt update
sudo apt install -y python3 python3-venv python3-pip curl

PIO_URL="https://raw.githubusercontent.com/platformio/platformio-core-installer"
curl -fsSL -o get-platformio.py ${PIO_URL}/master/get-platformio.py
python3 get-platformio.py

echo 'export PATH="$PATH:$HOME/.platformio/penv/bin"' >> ~/.bashrc
source ~/.bashrc

pio --version
```

ข้อมูลอัปเดต: PlatformIO Core เวอร์ชันปัจจุบันคือ 6.1.19 (เผยแพร่ กุมภาพันธ์ 2026) ใหม่กว่า 6.1.18 ที่ระบุในเอกสารต้นทาง สามารถอัปเดตด้วยคำสั่ง `pio upgrade`

6.3 GCC-AVR Toolchain — สำหรับเขียนระดับรีจิสเตอร์

สำหรับการเขียนโปรแกรมด้วย AVR LibC ซึ่งเข้าถึงรีจิสเตอร์ของชิปโดยตรง ต้องติดตั้งชุดเครื่องมือดังนี้

sudo apt install -y gcc-avr binutils-avr avr-libc avrdude	
เครื่องมือ	เหมาะกับงานแบบใด
arduino-cli	งาน Arduino Sketch ทั่วไป ติดตั้งง่าย เหมาะกับผู้เริ่มต้นและงานที่มีไฟล์ .ino เดิมอยู่แล้ว
PlatformIO (pio)	โปรเจกต์ขนาดใหญ่ หลายบอร์ด หลายเฟรมเวิร์ก มีระบบจัดการไลบรารีในไฟล์ platformio.ini ที่ทำซ้ำได้
avr-gcc + avr-libc	การเรียนสถาปัตยกรรมไมโครคอนโทรลเลอร์ระดับรีจิสเตอร์ เช่น Timer, ADC, PWM โดยไม่ผ่านไลบรารี Arduino

ส่วนที่ 7: ปฏิบัติการที่ 1 — Arduino Uno กับจอ LCD I2C

เริ่มจากโปรเจกต์ตัวอย่างของ Wokwi ที่มีไฟล์ตั้งค่าครบอยู่แล้ว เพื่อให้เห็นภาพรวมก่อนสร้างโปรเจกต์เอง

```
git clone https://github.com/wokwi/arduino-lcd-helloworld
cd arduino-lcd-helloworld
```

```
code . # เปิดโฟลเดอร์นี้ใน VS Code
```

เมื่อเปิดโปรเจกต์จะพบไฟล์ `diagram.json`, ไฟล์ `.ino` และ `wokwi.toml` จากนั้นคอมไพล์ด้วยคำสั่ง

```
arduino-cli compile \  
  --fqbn arduino:avr:uno \  
  --output-dir ./build/arduino.avr.uno/ .
```

คำอธิบายพารามิเตอร์: `--fqbn` (Fully Qualified Board Name) ระบุบอร์ดเป้าหมายในรูปแบบผู้ผลิต:สถาปัตยกรรม:รุ่น ส่วน `--output-dir` กำหนดที่เก็บไฟล์ผลลัพธ์ให้ตรงกับที่ระบุไว้ใน `wokwi.toml` เมื่อคอมไพล์เสร็จให้กด F1 แล้วเลือก "Wokwi: Start Simulator" เพื่อเริ่มจำลอง

7.1 แบบฝึกในคาบ — เพิ่มการนับและส่งข้อความออก Serial

ให้นักศึกษาแก้ไขโค้ดเดิมให้แสดงตัวนับบนจอ LCD พร้อมส่งข้อความออกทาง Serial Monitor ทุกหนึ่งวินาที โดยใช้เทคนิคจับเวลาแบบไม่บล็อกการทำงาน (non-blocking) ด้วย `millis()`

```
#include <LiquidCrystal_I2C.h>  
  
LiquidCrystal_I2C lcd(0x27, 16, 2); // ที่อยู่ I2C = 0x27, จอ 16x2  
  
const uint32_t INTERVAL = 1000; // คาบเวลา 1 วินาที  
uint32_t lastTick = 0; // เวลาที่ทำงานครั้งล่าสุด  
uint16_t counter = 0;  
  
void setup() {  
  Serial.begin(115200);  
  lcd.init();  
  lcd.backlight();  
  lcd.setCursor(0, 0);  
  lcd.print("RERU IoT Lab");  
}  
  
void loop() {  
  uint32_t now = millis();  
  if (now - lastTick >= INTERVAL) { // ถึงเวลาทำงานหรือยัง  
    lastTick = now;  
    counter++;  
  
    lcd.setCursor(0, 1);  
    lcd.print("Count: ");  
    lcd.print(counter);  
  
    Serial.print("tick = ");  
    Serial.println(counter);  
  }  
}
```

```
}
```

ประเด็นที่ต้องอภิปรายในชั้นเรียน: เหตุใดจึงใช้ `millis()` แทน `delay()`? — เพราะ `delay()` หยุดการทำงานของโปรแกรมทั้งหมด ทำให้ไม่สามารถอ่านปุ่มกดหรือทำงานอื่นระหว่างรอได้ ส่วน `millis()` ให้โปรแกรมวนทำงานต่อไปได้ ซึ่งเป็นรากฐานของการเขียนโปรแกรมฝังตัวที่ตอบสนองได้ดี

7.2 การตั้งค่า `tasks.json` เพื่อคอมไพล์ด้วยปุ่มลัด

การเพิ่มไฟล์ `.vscode/tasks.json` ช่วยให้สั่งคอมไพล์ด้วยปุ่มลัด `Ctrl+Shift+B` แทนการพิมพ์คำสั่งยาว ๆ ทุกครั้ง

```
{
  "version": "2.0.0",
  "tasks": [
    {
      "label": "Arduino: Compile (Uno)",
      "type": "shell",
      "command": "arduino-cli",
      "args": [
        "compile", "--verbose",
        "--fqbn", "arduino:avr:uno",
        "--output-dir", "${workspaceFolder}/build/arduino.avr.uno",
        "."
      ],
      "group": { "kind": "build", "isDefault": true },
      "problemMatcher": []
    }
  ]
}
```

ส่วนที่ 8: ปฏิบัติการที่ 2 — AVR LibC กับ ADC และ PWM

ปฏิบัติการนี้เขียนโปรแกรมระดับรีจิสเตอร์โดยไม่ใช้ไลบรารี Arduino เพื่อให้เข้าใจการทำงานภายในชิป ATmega328P โจทย์คือ อ่านค่าแรงดันจากโพเทนชิโอเมเตอร์ที่ขา A0 ด้วยวงจร ADC ขนาด 10 บิต แล้วนำค่าไปกำหนด Duty Cycle ของสัญญาณ PWM ที่ขา D9 เพื่อปรับความสว่างของ LED

8.1 โครงสร้างโปรเจกต์และคำสั่งคอมไพล์

```
mkdir -p uno_adc_pwm/src uno_adc_pwm/build && cd uno_adc_pwm
# สร้างไฟล์ src/main.c, wokwi.toml และ diagram.json

# คอมไพล์เป็นไฟล์ .elf
avr-gcc -mmcu=atmega328p -DF_CPU=16000000UL -Os -Wall \
  -o ./build/main.elf src/main.c
```

```
# แปลง .elf เป็นไฟล์เฟิร์มแวร์ .hex
avr-objcopy -O ihex -R .eeprom ./build/main.elf ./build/main.hex
```

คำอธิบายพารามิเตอร์สำคัญ: -mmcu=atmega328p ระบุชิปเป้าหมาย, -

DF_CPU=16000000UL บอกความถี่คริสตัล 16 MHz ซึ่งจำเป็นต่อการคำนวณเวลาให้ถูกต้อง, -Os ปรับให้ไฟล์ผลลัพธ์มีขนาดเล็กที่สุด (สำคัญมากเพราะหน่วยความจำโปรแกรมมีเพียง 32 KB)

8.2 โค้ดตัวอย่าง

```
#include <avr/io.h>

static void adc_init(void) {
    ADMUX = (1 << REFS0); // อ้างอิงแรงดัน AVcc, เลือกช่อง A0
    ADCSRA = (1 << ADEN) // เปิดใช้งานวงจร ADC
             | (1 << ADPS2) | (1 << ADPS1) | (1 << ADPS0); // ตัวหารความถี่ 128
}

static uint16_t adc_read(void) {
    ADCSRA |= (1 << ADSC); // สั่งเริ่มการแปลงสัญญาณ
    while (ADCSRA & (1 << ADSC)); // รอจนแปลงเสร็จ
    return ADC; // คืนค่า 0..1023
}

static void pwm_init(void) {
    DDRB |= (1 << PB1); // ขา D9 เป็นเอาต์พุต
    TCCR1A = (1 << COM1A1) | (1 << WGM11); // โหมด Fast PWM ขนาด 10 บิต
    TCCR1B = (1 << WGM13) | (1 << WGM12) | (1 << CS11);
    ICR1 = 1023; // ค่าสูงสุดของคาบสัญญาณ
}

int main(void) {
    adc_init();
    pwm_init();
    for (;;) {
        OCR1A = adc_read(); // ค่า ADC กำหนด Duty Cycle โดยตรง
    }
}
```

ในไฟล์ diagram.json ให้เพิ่มอุปกรณ์ wokwi-potentiometer ต่อขา SIG เข้ากับ A0 และเพิ่ม wokwi-logic-analyzer ต่อกับขา D9 เพื่อบันทึกการเปลี่ยนแปลงสัญญาณ

การวิเคราะห์รูปคลื่น: Logic Analyzer ใน Wokwi จะบันทึกผลเป็นไฟล์ VCD (Value Change Dump) ชื่อ wokwi-logic.vcd ซึ่งเปิดดูรูปคลื่นได้ด้วยโปรแกรม GTKWave — ให้นักศึกษาวัดคาบสัญญาณและ Duty Cycle จริงจากรูปคลื่น แล้วเทียบกับค่าที่คำนวณได้ทางทฤษฎี

ส่วนที่ 9: ปฏิบัติการที่ 3 — ESP32 กับ FreeRTOS

ปฏิบัติการสุดท้ายยกระดับสื่อบอร์ด ESP32 ซึ่งมีสองแกนประมวลผลและมีระบบปฏิบัติการเรียลไทม์ FreeRTOS ในตัว โจทย์คือสร้างสองทาสก์ที่ทำงานพร้อมกัน: ทาสก์หนึ่งกะพริบ LED อีกทาสก์อัปเดตข้อความบนจอ LCD

9.1 การสร้างโปรเจกต์ด้วย PlatformIO

```
mkdir -p esp32-lcd-i2c && cd esp32-lcd-i2c
pio project init --board esp32dev
```

แก้ไขไฟล์ platformio.ini ให้ระบุไลบรารีที่ต้องใช้ ระบบจะดาวน์โหลดให้อัตโนมัติตอนคอมไพล์

```
[env:esp32dev]
platform = espressif32
board = esp32dev
framework = arduino
monitor_speed = 115200

lib_deps =
    marcoschwartz/LiquidCrystal_I2C@^1.1.4
```

9.2 โค้ดตัวอย่างการสร้างทาสก์

```
#include <Arduino.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

LiquidCrystal_I2C lcd(0x27, 16, 2);
#define LED_PIN 2 // LED บนบอร์ด ESP32 DevKit ส่วนใหญ่

// ทาสก์ที่ 1: กะพริบ LED ทุก 500 มิลลิวินาที
void taskBlink(void *param) {
    pinMode(LED_PIN, OUTPUT);
    bool state = false;
    for (;;) {
        digitalWrite(LED_PIN, state = !state);
        vTaskDelay(pdMS_TO_TICKS(500)); // คั่น CPU ให้ทาสก์อื่นระหว่างรอ
    }
}

// ทาสก์ที่ 2: อัปเดตตัวนับบนจอ LCD ทุก 1 วินาที
void taskDisplay(void *param) {
    uint16_t count = 0;
    for (;;) {
        lcd.setCursor(0, 1);
        lcd.print("Count: ");
        lcd.print(count++);
    }
}
```

```

    vTaskDelay(pdMS_TO_TICKS(1000));
}
}

void setup() {
    Serial.begin(115200);
    Wire.begin(21, 22);                // SDA = GPIO21, SCL = GPIO22
    lcd.init();
    lcd.backlight();
    lcd.setCursor(0, 0);
    lcd.print("FreeRTOS Ready");

    //          พังก์ชัน      ชื่อ      สตริง      พารามิเตอร์ ลำดับ ตัวชี้ แกน
    xTaskCreatePinnedToCore(taskBlink, "blink", 2048, NULL, 1, NULL, 0);
    xTaskCreatePinnedToCore(taskDisplay, "lcd", 2048, NULL, 1, NULL, 1);
}

void loop() {
    // บล๊อคว่างได้ เพราะงานทั้งหมดถูกย้ายไปอยู่ในทาสก์แล้ว
}

```

ประเด็นสำคัญที่ต้องอธิบาย: `vTaskDelay()` ต่างจาก `delay()`

ตรงที่มันคืนเวลาประมวลผลให้ทาสก์อื่นระหว่างรอ ทำให้ทั้งสองทาสก์ทำงานคู่ขนานได้จริง

ส่วนพารามิเตอร์ตัวสุดท้ายของ `xTaskCreatePinnedToCore()` คือหมายเลขแกนประมวลผล (0 หรือ 1) ซึ่งเป็นความสามารถเฉพาะของ ESP32 ที่มีสองแกน

คอมไพล์ด้วยคำสั่ง `pio run` ไฟล์ `.bin` และ `.elf` จะถูกเก็บไว้ที่ `.pio/build/esp32dev/` ตรงตามที่ระบุใน `wokwi.toml`

ส่วนที่ 10: การบูรณาการ Generative AI ในงานพัฒนาระบบฝังตัว

10.1 หลักคิด: AI เป็นผู้ช่วยดีบั๊ก ไม่ใช่ผู้เขียนแทน

งานระบบฝังตัวมีลักษณะเฉพาะที่ทำให้ต้องระงับการใช้ AI มากกว่างานซอฟต์แวร์ทั่วไป เพราะ โค้ดที่ "ดูเหมือนถูก" อาจทำให้ฮาร์ดแวร์จริงเสียหายได้ เช่น การตั้งค่าขาเป็นเอาต์พุตแล้วต่อเข้ากับแหล่งจ่ายไฟ หรือการดึงกระแสเกินพิกัดของขา นอกจากนี้โมเดล AI มักสับสนระหว่างชิปตระกูลต่าง ๆ (เช่น ใช้ชื่อรีจิสเตอร์ของ ATmega328P กับ ESP32) ดังนั้นแนวทางในรายวิชานี้คือ

- ให้ AI อธิบาย ไม่ใช่ให้ AI เขียน — ใช้ AI แปลความหมายของ error message และอธิบายว่ารีจิสเตอร์แต่ละบิตทำอะไร
- ตรวจสอบกับ datasheet เสมอ — ทุกคำตอบที่เกี่ยวกับรีจิสเตอร์ ขา หรือค่าไฟฟ้า ต้องยืนยันกับเอกสารทางการของผู้ผลิต

- ทดสอบในตัวจำลองก่อนเสมอ — Wokwi
ทำหน้าที่เป็นด่านตรวจสอบขั้นแรกที่ไม่มีต้นทุนความเสียหาย

10.2 Workflow การใช้ AI ในปฏิบัติการนี้ (5 ขั้นตอน)

ขั้น	กิจกรรม	บทบาทของ AI
1	ออกแบบวงจรและเขียนโค้ดเอง	ไม่ใช่ AI — ต้องคิดโครงสร้างโปรแกรมและเลือกอุปกรณ์ด้วยตนเอง
2	แปลง error จากคอมไพเลอร์	วางข้อความ error พร้อมบริบท (บอร์ด เฟรมเวิร์ก บรรทัดที่เกี่ยวข้อง) ให้ AI อธิบายสาเหตุ
3	ตรวจทานโค้ด (code review)	ขอให้ AI ชี้จุดเสี่ยง เช่น การใช้ delay() ที่บล็อกการทำงาน ตัวแปรที่ควรเป็น volatile หรือขนาดสแต็กของทาสก์
4	ร่างไฟล์ diagram.json	ให้ AI ร่างโครงสร้าง JSON ตามรายการอุปกรณ์ที่นักศึกษากำหนด แล้วให้นักศึกษาตรวจแก้ชื่อตัวเอง
5	ยืนยันด้วยการจำลองจริง	ไม่ใช่ AI — ผลการจำลองใน Wokwi และ datasheet คือคำตอบสุดท้าย พร้อมบันทึก AI Log

10.3 ตัวอย่าง Prompt ที่ใช้ได้จริง

Prompt 1 — วิเคราะห์ข้อความผิดพลาด:

"ผมคอมไพล์โปรเจกต์ Arduino สำหรับบอร์ด Arduino Uno ด้วย arduino-cli แล้วได้ error ต่อไปนี้ [วาง error ทั้งหมด] นี่เป็นโค้ดส่วนที่เกี่ยวข้อง [วางโค้ด] ช่วยอธิบายว่าสาเหตุคืออะไร เกิดจากบรรทัดไหน และมีทางเลือกที่แบบ พร้อมข้อดีข้อเสียของแต่ละแบบ โดยไม่ต้องเขียนโค้ดใหม่ให้ทั้งหมด"

Prompt 2 — อธิบายรีจิสเตอร์ระดับบิต:

"อธิบายว่าคำสั่ง ADMUX = (1 << REFS0); ในชิป ATmega328P ทำอะไร ให้แจกแจงทีละบิตว่าบิตไหนถูกตั้งค่าและมีผลอย่างไรต่อวงจร ADC พร้อมระบุหน้าที่ควรอ่านเพิ่มเติมใน datasheet"

Prompt 3 — ตรวจสอบโค้ดเชิงระบบฝังตัว:

"ทำหน้าที่เป็นวิศวกรระบบฝังตัวที่ตรวจสอบโค้ด ตรวจสอบโค้ด ESP32 + FreeRTOS นี้ใน 4 ประเด็น: (1) การใช้หน่วยเวลาที่บล็อกการทำงาน (2) ตัวแปรที่ใช้ร่วมกันระหว่างทาสก์โดยไม่มีการป้องกัน (3) ขนาดสแต็กที่อาจไม่พอ (4) การตั้งค่าขา GPIO ที่เสี่ยง — ซึ่งเป็นรายการที่ห้ามเขียนโค้ดใหม่ให้ทั้งหมด"

Prompt 4 — ร่างวงจรเป็น JSON:

"ช่วยร่างไฟล์ diagram.json ของ Wokwi สำหรับวงจรที่ประกอบด้วย: บอร์ด Arduino Uno, ปุ่มกด 1 ตัวต่อขา D2 แบบ pull-up ภายใน, LED 1 ดวงพร้อมตัวต้านทาน 220 โอห์มที่ขา D8 ให้ใช้ชนิดอุปกรณ์ที่มีจริงใน elements.wokwi.com เท่านั้น และอธิบายการเชื่อมต่อแต่ละเส้นด้วยว่าเชื่อมขาใดกับขาใด"

ข้อสังเกตสำคัญ: AI มักสร้างชื่ออุปกรณ์หรือชื่อขาที่ไม่มีอยู่จริง (hallucination) การกำหนดในคำสั่งว่า "ให้ใช้เฉพาะอุปกรณ์ที่มีจริง" ช่วยลดปัญหาได้บ้าง แต่ไม่หมด — นักศึกษาต้องเปิดหน้า elements.wokwi.com ตรวจสอบทุกครั้ง และผลการจำลองที่ล้มเหลวคือหลักฐานที่ชัดที่สุดว่าคำตอบผิด

ส่วนที่ 11: จริยธรรมและความเชื่อสัตย์ทางวิชาการ

- **เปิดเผยการใช้ AI (AI Log)** — ระบุเครื่องมือที่ใช้ prompt ที่ใช้ และส่วนใดของงานที่ AI มีส่วนช่วย แแนบท้ายรายงานทุกครั้ง
- **ต้องอธิบายโค้ดของตนเองได้ทุกบรรทัด** — ในการสอบปฏิบัติ ผู้สอนจะสุ่มถามความหมายของโค้ด การส่งงานที่อธิบายไม่ได้ถือว่าไม่ผ่านแม้โปรแกรมจะทำงานถูกต้อง
- **เคารพสัญญาอนุญาตของโค้ดที่นำมาใช้** — โปรเจกต์ตัวอย่างและไลบรารีต่าง ๆ มีสัญญาอนุญาตกำกับ (เช่น MIT, GPL, CC BY-SA) ต้องอ้างอิงแหล่งที่มาและปฏิบัติตามเงื่อนไข
- **ความปลอดภัยเมื่อย้ายไปฮาร์ดแวร์จริง** — ก่อนต่อวงจรจริงต้องตรวจสอบขั้วแหล่งจ่าย ค่าตัวต้านทานจำกัดกระแส และพิกัดกระแสของขา I/O เสมอ ผลการจำลองที่ผ่านไม่ใช่การรับประกันความปลอดภัย

ส่วนที่ 12: กิจกรรมในชั้นเรียน (In-class Activities)

- **กิจกรรม 1 — แข่งตั้งค่าสภาพแวดล้อม (30 นาที):** จับคู่ติดตั้ง VS Code + Wokwi Extension + arduino-cli แล้วรันโปรเจกต์ตัวอย่างให้จำลองได้สำเร็จ คู่ที่เสร็จก่อนช่วยคู่ที่ติดปัญหา (peer support) จบด้วยการรวบรวมปัญหาที่พบขึ้นกระดานเป็นคลังความรู้ของห้อง
- **กิจกรรม 2 — อ่านวงจรจาก JSON (25 นาที):** แจกไฟล์ diagram.json ที่ไม่มีรูปประกอบให้นักศึกษาวาดผังวงจรลงกระดาษจากข้อมูล JSON ล้วน แล้วเปิดใน Wokwi เทียบว่าตรงหรือไม่ — ฝึกทักษะการอ่านโครงสร้างข้อมูล
- **กิจกรรม 3 — ล่าบั๊ก (Bug Hunt) (30 นาที):** ผู้สอนแจกโปรเจกต์ที่มีข้อผิดพลาดฝังไว้ 3 จุด (เช่น พารามิเตอร์ใน wokwi.toml ผิด, ที่อยู่ I2C ผิด, ลืมตั้งค่าเป็นเอาต์พุต) ให้แข่งกันหาและแก้ไขให้ครบ พร้อมบันทึกวิธีวินิจฉัย
- **กิจกรรม 4 — AI Debug Lab (40 นาที):** ใช้ Prompt 1 และ 3 ในส่วนที่ 10 กับบั๊กจากกิจกรรม 3 แล้วประเมินว่า AI ช่วยได้จริงกี่จุด ตอบผิดหรือมั่วกี่จุด บันทึกผลการเปรียบเทียบ — ปิดท้ายด้วยการอภิปรายว่าเมื่อใดควรเชื่อ AI และเมื่อใดต้องเปิด datasheet
- **กิจกรรม 5 — Show & Tell (25 นาที):** แต่ละกลุ่มสาธิตโปรเจกต์ต้นฉบับของตน 3 นาที โดยต้องอธิบายเหตุผลของการเลือกอุปกรณ์และโครงสร้างโปรแกรม เพื่อนโหวต "โปรเจกต์ที่อธิบายได้ชัดเจนที่สุด"

ส่วนที่ 13: แบบฝึกหัด/คำถามท้ายบท (พร้อมแนวคำตอบ)

13.1 คำถามทบทวน

1. ไฟล์ wokwi.toml และ diagram.json ทำหน้าที่ต่างกันอย่างไร? แนวตอบ: wokwi.toml ระบุตำแหน่งไฟล์เฟิร์มแวร์ (.hex/.bin) และ .elf ที่ได้จากการคอมไพล์ ส่วน diagram.json อธิบายวงจร คือรายการอุปกรณ์และการเชื่อมต่อระหว่างขา
2. เมื่อใช้ Wokwi ผ่าน VS Code การคอมไพล์และการจำลองเกิดขึ้นที่ใด? แนวตอบ: คอมไพล์เกิดบนเครื่องผู้ใช้ด้วย toolchain ที่ติดตั้งเอง ส่วนการจำลองยังทำงานผ่านบริการของ Wokwi จึงต้องเชื่อมต่ออินเทอร์เน็ต
3. เหตุใดบอร์ด Uno ใช้ไฟล์ .hex แต่ ESP32 ใช้ .bin? แนวตอบ: เป็นรูปแบบไฟล์เฟิร์มแวร์ตามสถาปัตยกรรมและ toolchain ของแต่ละตระกูลชิป — AVR ใช้ Intel HEX ส่วน ESP32 ใช้ไบนารีที่มีโครงสร้างพาร์ทิชันของตนเอง ส่วนไฟล์ .elf ใช้ประกอบการดีบั๊กทั้งสองกรณี

4. พารามิเตอร์ `--fqbn` ในคำสั่ง `arduino-cli compile` หมายถึงอะไร ยกตัวอย่าง 1 ค่า แนวตอบ: Fully Qualified Board Name คือชื่อบอร์ดเป้าหมายในรูปแบบ ผู้ผลิต:สถาปัตยกรรม:รุ่น เช่น `arduino:avr:uno`
5. เปรียบเทียบข้อดีของ `arduino-cli` กับ PlatformIO มาอย่างละ 1 ข้อ แนวตอบ: `arduino-cli` ติดตั้งง่ายและเหมาะกับ Arduino Sketch เดิม; PlatformIO จัดการหลายบอร์ด/หลายเฟรมเวิร์ก และประกาศไลบรารีในไฟล์ `platformio.ini` ทำให้ทำซ้ำสภาพแวดล้อมได้
6. เหตุใด `vTaskDelay()` จึงเหมาะกว่า `delay()` ในโปรแกรม FreeRTOS? แนวตอบ: `vTaskDelay()` คำนวณเวลาประมวลผลให้ทำสักรั้วรอ ทำให้ทำสักรั้วทำงานคู่ขนานได้จริง ขณะที่ `delay()` ยึดครองการทำงานไว้
7. ยกข้อจำกัดของการทดสอบด้วย Simulator มา 2 ข้อ แนวตอบ: ฮาร์ดแวร์บางส่วนอาจยังไม่รองรับหรือไม่ครอบคลุมทั้งหมด และตัวจำลองไม่สะท้อนปัจจัยทางกายภาพ เช่น สัญญาณรบกวน กระแสจริงที่อุปกรณ์ดึง หรือปัญหาการต่อกราวด์
8. (วิเคราะห์) นักศึกษาคอมไพล์สำเร็จแต่กดจำลองแล้ว Wokwi แจ้งว่าไม่พบไฟล์เฟิร์มแวร์ จงระบุสาเหตุที่เป็นไปได้ 3 ข้อ พร้อมวิธีตรวจสอบทีละข้อ แนวตอบ: (1) พาย์ใน `wokwi.toml` ไม่ตรงกับ `--output-dir` ตรวจสอบโดยเทียบชื่อโฟลเดอร์จริง (2) ชื่อไฟล์ผลลัพธ์ไม่ตรง เพราะ `arduino-cli` ตั้งชื่อตามชื่อโฟลเดอร์โปรเจกต์ (3) คอมไพล์ไม่สำเร็จจริงแต่มองข้าม error ตรวจสอบโดยดูรหัสสถานะและอ่านบรรทัดสุดท้ายของผลลัพธ์

13.2 แบบฝึกหัดปฏิบัติ (ส่งเป็นชิ้นงาน)

- **แบบฝึกหัด A — โปรเจกต์ต้นฉบับ:** ออกแบบและสร้างโปรเจกต์ Wokwi ที่มีอุปกรณ์น้อยที่สุดอย่างน้อย 1 ชิ้นและเอาต์พุตอย่างน้อย 2 ชิ้น ส่งเป็นโฟลเดอร์ที่มี `diagram.json`, `wokwi.toml`, ซอร์สโค้ด, ไฟล์ตั้งค่าการคอมไพล์ และ README อธิบายวงจรพร้อมภาพผลการจำลอง (แนะนำให้ส่งเป็นรีโพสิทอรี Git)
- **แบบฝึกหัด B — รายงานการติบักด้วย AI:** บันทึกการแก้ปัญหาอย่างน้อย 3 ครั้งระหว่างทำแบบฝึกหัด A ในรูปแบบตาราง: อาการที่พบ / สมมติฐานของตนเอง / prompt ที่ใช้ถาม AI / คำตอบของ AI / ผลการตรวจสอบจริง (ถูกหรือผิด) / วิธีแก้สุดท้าย ปิดท้ายด้วยข้อสรุปครั้งหน้าว่า AI ช่วยงานลักษณะใดได้ดีและงานลักษณะใดที่ยังเชื่อไม่ได้

ส่วนที่ 14: เกณฑ์การประเมิน (Assessment Rubric)

เกณฑ์	ดีเยี่ยม (4)	ดี (3)	พอใช้ (2)	ปรับปรุง
-------	--------------	--------	-----------	----------

การตั้งค่าโปรเจกต์	ไฟล์ครบ พาสถูกต้อง จำลองได้ทันทีจากเครื่องอื่น	ไฟล์ครบ จำลองได้	ต้องแก้พาสเล็กน้อยจึงรันได้	รันไม่
การออกแบบวงจร (diagram.json)	อุปกรณ์เหมาะสม ต่อถูกหลักไฟฟ้า มีตัวต้านทานจำกัดกระแสครบ	ต่อถูกต้อง ครบตามโจทย์	มีจุดต่อที่ไม่เหมาะสม	วงจรผิด
คุณภาพโค้ด	โครงสร้างชัด ไม่บล็อกการทำงาน ตั้งชื่อสื่อความหมาย มีคอมเมนต์ที่จำเป็น	ทำงานถูกต้อง อ่านเข้าใจได้	ทำงานได้แต่โครงสร้างสับสน	ทำงาน
การวิเคราะห์ปัญหา	วินิจฉัยเป็นระบบ มีสมมติฐานและการทดสอบทีละขั้น	ระบุสาเหตุได้ถูกต้อง	แก้ได้แต่อธิบายสาเหตุไม่ชัด	แก้แบบ
การใช้ AI อย่างมีวิจารณญาณ	ตรวจสอบคำตอบ AI กับ datasheet/ผลจำลองทุกครั้ง ระบุจุดที่ AI ผิดได้	ใช้ AI ช่วยและตรวจสอบผล	รับคำตอบ AI มาใช้เกือบทั้งหมด	ไม่มีร้อ
จริยธรรมและเอกสาร (AI Log)	บันทึกครบ อ้างอิงแหล่งโค้ด สะท้อนคิดลึก	บันทึกครบถ้วน	บันทึกไม่ครบ	ไม่เปิด

การให้น้ำหนัก (ปรับได้ตามผู้สอน): ทักษะปฏิบัติหลัก (เกณฑ์ 1–3) 60% / การวิเคราะห์และการใช้ AI (เกณฑ์ 4–5) 25% / เอกสารและจริยธรรม (เกณฑ์ 6) 15%

ส่วนที่ 15: เอกสารอ้างอิงและแหล่งค้นคว้าเพิ่มเติม

- IoT Engineering Education. แนะนำการใช้งาน VS Code IDE + Wokwi. <https://iot-kmutnb.github.io/blogs/wokwi/wokwi-vscode/> (เผยแพร่ 7 ม.ค. 2026, สัญญาอนุญาต CC BY-SA 4.0) — เอกสารต้นทางหลักของบทเรียนนี้
- Wokwi Documentation. Getting Started with Wokwi for VS Code. <https://docs.wokwi.com/vscode/getting-started>
- Wokwi Elements. รายการอุปกรณ์ที่รองรับทั้งหมด. <https://elements.wokwi.com/>
- Arduino. Arduino CLI Documentation. <https://docs.arduino.cc/arduino-cli/> (เวอร์ชันล่าสุด 1.5.1, มิ.ย. 2026)

- PlatformIO. PlatformIO Core Documentation. <https://docs.platformio.org/> (เวอร์ชันล่าสุด 6.1.19, ก.พ. 2026)
- Microchip Technology. ATmega328P Datasheet — เอกสารอ้างอิงรีจิสเตอร์สำหรับปฏิบัติการที่ 2
- Espressif Systems. ESP32 Technical Reference Manual และ FreeRTOS API Reference — สำหรับปฏิบัติการที่ 3

หมายเหตุสำหรับอาจารย์ผู้สอน: เอกสารนี้จัด heading hierarchy พร้อมสารบัญอัตโนมัติแล้ว

(คลิกขวาที่สารบัญ > Update Field ใน Word)

แนะนำให้แทรกภาพหน้าจอขั้นตอนการติดตั้งจากเครื่องของผู้สอนเอง

เพื่อให้ตรงกับเวอร์ชันที่ใช้ในห้องปฏิบัติการ

และควรทดสอบขั้นตอนติดตั้งทั้งหมดบนเครื่องห้องแล็บก่อนสอนจริงอย่างน้อยหนึ่งสัปดาห์

เนื่องจากเวอร์ชันของเครื่องมือเปลี่ยนแปลงบ่อย